

SQLite设计

一、SQLite 的职责边界

SQLite 应该存：

用户做题记录
用户错因
用户对知识点、方法、题型、解题思想的掌握程度
最近学习上下文摘要
当前会话状态
Agent 当前 **workflow** 状态
工具调用日志
待写回状态
复习计划
短期记忆
系统配置
异常恢复信息

SQLite 不应该存：

完整题库
完整标准解析库
完整相似题网络
知识点资产定义
方法资产定义
大规模 **embedding**
长期稳定题目资产

这些属于 PostgreSQL。

SQLite 里可以保存 `question_id`、`knowledge_point_id`、`method_id`，但这些 ID 指向 PostgreSQL 中的资产。

也就是说：

SQLite 记录：用户在 `question_id = Q123` 上答错了，错因是 `method_selection_error`。
PostgreSQL 记录：`Q123` 的题干、答案、解析、知识点、方法、相似题关系。

二、SQLite 总体分层

SQLite 建议分成四层：

1. 学习行为层
2. 学习状态层
3. Agent 记忆层
4. 系统运行层

对应表：

学习行为层：

- learning_session
- conversation_turn
- question_attempt
- review_event

学习状态层：

- mastery_state
- mastery_event
- error_pattern_state
- review_schedule

Agent 记忆层：

- context_summary
- agent_memory_item
- active_context_ref

系统运行层：

- agent_runtime_state
- workflow_run
- tool_call_log
- pending_state_delta
- system_kv
- local_event_log

核心思想是：

行为记录是事实。

状态表是快照。

event_log 是可回放历史。

context_summary 是给 Agent 拼 Prompt 用的压缩记忆。

三、SQLite 建库基础配置

建议初始化时使用：

```
pragma foreign_keys = on;
pragma journal_mode = wal;
pragma synchronous = normal;
pragma busy_timeout = 5000;
```

建一个迁移表：

```
create table if not exists schema_migration (
    version integer primary key,
    name text not null,
    applied_at text not null default current_timestamp
);
```

SQLite 早期可以单用户运行，但表结构最好预留 `user_id`。这样以后迁移到多人系统不会重构太痛苦。

四、用户与会话表

1. 用户本地表

即使早期只有一个用户，也建议保留。

```
create table user_profile (
    user_id text primary key,

    display_name text,

    current_subject text,
    current_stage text,

    preference_json text not null default '{}',

    created_at text not null default current_timestamp,
    updated_at text not null default current_timestamp
);
```

这里的 `preference_json` 可以存：

```
{  
  "explanation_style": "先整体框架，再细节推导",  
  "preferred_difficulty": 3,  
  "show_solution_steps": true,  
  "show_method_comparison": true  
}
```

注意：这不是长期个人隐私画像，而是学习系统运行所需的偏好配置。

2. 学习会话表 `learning_session`

每次打开 Iris 或进入一次学习任务，创建一个 session。

```
create table learning_session (  
  id text primary key,  
  
  user_id text not null references user_profile(user_id) on delete cascade,  
  
  session_type text not null,  
  -- new_question / review / diagnosis / free_chat / mixed  
  
  subject text,  
  topic text,  
  
  status text not null default 'active',  
  -- active / completed / interrupted / failed  
  
  started_at text not null default current_timestamp,  
  ended_at text,  
  
  summary_text text,  
  
  metadata_json text not null default '{}'  
);
```

作用：

- 记录一次学习过程
- 给上下文压缩提供边界
- 给复习调度提供来源
- 给 Agent 恢复上下文

3. 对话轮次表 conversation_turn

只存必要信息，不要无限塞完整上下文进 Prompt。

```
create table conversation_turn (  
  id text primary key,  
  
  session_id text not null references learning_session(id) on delete  
cascade,  
  user_id text not null references user_profile(user_id) on delete cascade,  
  
  role text not null,  
  -- user / assistant / tool / system  
  
  content_text text not null,  
  
  content_type text not null default 'text',  
  -- text / question / answer / tool_result / render_json  
  
  token_estimate integer,  
  
  related_question_id text,  
  related_attempt_id text,  
  
  created_at text not null default current_timestamp,  
  
  metadata_json text not null default '{}'  
);
```

这张表用于保留原始对话，但主 Agent 不应该每次都读取全部历史。真正进 Prompt 的应该是 `context_summary`。

五、做题记录表 question_attempt

这是 SQLite 最重要的事实表之一。

```
create table question_attempt (  
  id text primary key,  
  
  user_id text not null references user_profile(user_id) on delete cascade,  
  session_id text references learning_session(id) on delete set null,  
  
  question_id text not null,
```

```

-- 指向 PostgreSQL.question_asset.id

source_type text not null default 'pgsql',
-- pgsql / generated_draft / user_input

user_answer_text text,

is_correct integer,
-- 1 正确, 0 错误, null 未判定

score real,
-- 0 到 1

attempt_status text not null default 'submitted',
-- viewed / submitted / checked / abandoned / skipped

time_spent_seconds integer,

difficulty_level integer,

main_error_type text,
-- knowledge_gap / method_selection_error / calculation_error /
misunderstanding / expression_error / careless / unknown

error_detail_text text,

confidence_self_report real,
-- 用户自评, 0 到 1, 可选

created_at text not null default current_timestamp,
checked_at text,

metadata_json text not null default '{}'
);

```

这张表只记录事实：

用户做了哪道题
 答了什么
 对不对
 错因是什么
 用了多久
 属于什么任务

不要在这里直接写“用户掌握了极限”。掌握度属于 `mastery_state`。

六、题目关联快照表 `question_attempt_tag`

做题记录需要绑定知识点、方法、题型、结构。虽然 PostgreSQL 有完整映射，但 SQLite 需要保存当时的快照，防止以后 PostgreSQL 标签变化导致历史状态解释不一致。

```
create table question_attempt_tag (  
    id text primary key,  
  
    attempt_id text not null references question_attempt(id) on delete  
cascade,  
  
    tag_type text not null,  
    -- knowledge_point / method / question_type / structure /  
    thinking_pattern  
  
    tag_id text,  
    -- PostgreSQL 中的 knowledge_point_id / method_id 等  
  
    tag_name text not null,  
  
    role text not null default 'secondary',  
    -- primary / secondary / prerequisite / hidden  
  
    weight real not null default 1.0,  
  
    confidence real not null default 1.0,  
  
    created_at text not null default current_timestamp  
);
```

这张表非常关键。因为状态引擎需要知道：

这次答题影响哪些知识点？
影响哪些方法？
影响哪类题型？
暴露哪种解题思想问题？

其中 `thinking_pattern` 是你提到的“某类题型思想”的核心。

例如：

```
tag_type = thinking_pattern  
tag_name = 主动构造上下界
```

```
tag_name = 识别夹逼结构
tag_name = 从目标反推变形
tag_name = 构造辅助函数
tag_name = 先分类再讨论
tag_name = 将几何条件代数化
```

这类标签可以先在 SQLite 里作为状态标签运行，成熟后再沉淀进 PostgreSQL 的 `method_asset` 或 `question_structure_feature`。

七、掌握状态表 mastery_state

这是 SQLite 的核心状态表。

不要为知识点、方法、题型、思想分别建四套重复表。建议用一个泛化表：

```
create table mastery_state (
    id text primary key,

    user_id text not null references user_profile(user_id) on delete cascade,

    entity_type text not null,
    -- knowledge_point / method / question_type / structure /
    thinking_pattern

    entity_id text,
    -- 如果来自 PostgreSQL，则保存 PostgreSQL ID；如果是本地临时标签，可以为空

    entity_name text not null,

    mastery_score real not null default 0.0,
    -- 0 到 1

    confidence_score real not null default 0.0,
    -- 系统对 mastery_score 的置信度，0 到 1

    exposure_count integer not null default 0,
    correct_count integer not null default 0,
    wrong_count integer not null default 0,

    recent_score real,
    -- 最近几次表现的滑动分数

    state_label text not null default 'unseen',
    -- unseen / exposed / learning / weak / reviewing / stable / mastered
```

```
last_seen_at text,  
last_success_at text,  
last_failure_at text,  
  
next_review_at text,  
  
decay_rate real not null default 0.05,  
  
updated_at text not null default current_timestamp,  
  
unique(user_id, entity_type, entity_name)  
);
```

状态标签建议这样定义：

unseen	未接触
exposed	接触过，但证据不足
learning	正在学习
weak	薄弱
reviewing	复习中
stable	稳定掌握
mastered	高置信掌握

mastery_score 是掌握度， **confidence_score** 是系统判断的置信度。两者必须分开。

例如：

用户做对 1 道题：
mastery_score 可以升到 0.65
confidence_score 可能只有 0.25

用户连续做对 8 道不同结构题：
mastery_score 可以 0.85
confidence_score 可以 0.8

这比简单记录“掌握 / 未掌握”更可靠。

八、掌握变化事件表 mastery_event

mastery_state 是当前快照， **mastery_event** 是变化过程。

```
create table mastery_event (  
  id text primary key,  
  
  user_id text not null references user_profile(user_id) on delete cascade,  
  
  mastery_state_id text references mastery_state(id) on delete set null,  
  
  attempt_id text references question_attempt(id) on delete set null,  
  review_event_id text,  
  
  entity_type text not null,  
  entity_id text,  
  entity_name text not null,  
  
  old_mastery_score real,  
  new_mastery_score real,  
  
  old_confidence_score real,  
  new_confidence_score real,  
  
  old_state_label text,  
  new_state_label text,  
  
  delta_reason text not null,  
  -- correct_answer / wrong_answer / review_success / review_failed /  
  manual_adjust / agent_suggestion  
  
  evidence_text text,  
  
  created_at text not null default current_timestamp,  
  
  metadata_json text not null default '{}'  
);
```

为什么需要这个表？

因为你后面一定会问：

系统为什么认为我“夹逼法”薄弱？
为什么今天推这道复习题？
为什么掌握度下降了？

如果只有当前状态，没法解释。

有了 `mastery_event`，就可以回放状态变化。

九、错因状态表 error_pattern_state

错因不是一次性记录。用户可能反复出现同类错误。

```
create table error_pattern_state (  
    id text primary key,  
  
    user_id text not null references user_profile(user_id) on delete cascade,  
  
    error_type text not null,  
    -- knowledge_gap / method_selection_error / calculation_error /  
    misunderstanding / expression_error / careless  
  
    error_pattern_name text not null,  
    -- 不会主动构造上下界 / 忘记讨论参数范围 / 把充分条件当必要条件  
  
    related_entity_type text,  
    -- knowledge_point / method / question_type / thinking_pattern  
  
    related_entity_name text,  
  
    occurrence_count integer not null default 0,  
  
    severity_score real not null default 0.0,  
    -- 0 到 1  
  
    last_occured_at text,  
  
    resolved_score real not null default 0.0,  
    -- 0 表示未解决, 1 表示基本解决  
  
    status text not null default 'active',  
    -- active / improving / resolved / ignored  
  
    updated_at text not null default current_timestamp,  
  
    unique(user_id, error_pattern_name, related_entity_name)  
);
```

例子:

```
error_pattern_name = 不会主动构造上下界  
related_entity_type = thinking_pattern
```

```
related_entity_name = 识别夹逼结构
severity_score = 0.72
```

这样复习 Agent 就能知道：

不要只推“极限题”
而要推“需要主动构造上下界的极限题”

这就是你的系统和普通题库的差异。

十、复习调度表 review_schedule

这是复习系统的核心。

```
create table review_schedule (
    id text primary key,

    user_id text not null references user_profile(user_id) on delete cascade,

    target_type text not null,
    -- question / knowledge_point / method / question_type / thinking_pattern
    / error_pattern

    target_id text,
    target_name text not null,

    priority_score real not null default 0.0,

    due_at text not null,

    interval_days real not null default 1.0,

    ease_factor real not null default 2.5,

    review_count integer not null default 0,

    last_review_at text,
    last_result text,
    -- success / fail / partial / skipped

    status text not null default 'scheduled',
    -- scheduled / due / completed / suspended / cancelled
```

```
created_at text not null default current_timestamp,  
updated_at text not null default current_timestamp,  
  
metadata_json text not null default '{}'  
);
```

复习对象不应该只限于题目。它可以是：

某道旧题
某个知识点
某个方法
某类题型
某种解题思想
某种错因模式

例如：

```
target_type = thinking_pattern  
target_name = 主动构造上下界  
due_at = 明天  
priority_score = 0.88
```

这才是更适合 Agent 教学系统的复习调度。

十一、复习事件表 review_event

每次复习都记录下来。

```
create table review_event (  
    id text primary key,  
  
    user_id text not null references user_profile(user_id) on delete cascade,  
    session_id text references learning_session(id) on delete set null,  
  
    schedule_id text references review_schedule(id) on delete set null,  
  
    review_type text not null,  
    -- old_question_review / same_knowledge_new_question / method_transfer /  
    error_pattern_review  
  
    target_type text not null,  
    target_id text,  
    target_name text not null,
```

```

    result text,
    -- success / fail / partial / skipped

    score real,

    user_feedback_text text,

    agent_review_summary text,

    next_review_at text,

    created_at text not null default current_timestamp,

    metadata_json text not null default '{}';
);

```

复习之后，系统应该同时更新：

```

review_event
review_schedule
mastery_state
mastery_event
context_summary

```

十二、Agent 上下文摘要表 context_summary

这是 Agent 的短期记忆来源。

它不是完整历史，而是压缩后的上下文。

```

create table context_summary (
    id text primary key,

    user_id text not null references user_profile(user_id) on delete cascade,

    session_id text references learning_session(id) on delete cascade,

    summary_scope text not null,
    -- turn / session / topic / profile / workflow

    summary_text text not null,

```

```

structured_json text not null default '{}',

source_turn_start_id text,
source_turn_end_id text,

token_estimate integer,

confidence real not null default 1.0,

status text not null default 'active',
-- active / superseded / deprecated

created_at text not null default current_timestamp,
updated_at text not null default current_timestamp
);

```

`structured_json` 建议长这样：

```

{
  "current_subject": "高等数学",
  "current_topic": "极限",
  "recent_focus": ["数列极限", "夹逼估计"],
  "weak_points": [
    {
      "type": "thinking_pattern",
      "name": "主动构造上下界",
      "evidence": "用户在两道夹逼题中需要提示才能构造上下界"
    }
  ],
  "active_question_ids": ["Q123"],
  "open_loops": [
    "需要再做一道同方法但题面变化更大的题"
  ]
}

```

这张表用于拼 Prompt。

主 Agent 启动时，不应该读全部 `conversation_turn`，而应该读：

```

最新 session_summary
最新 topic_summary
最近 N 条 active memory
当前 due review

```

当前 weak mastery

当前 active workflow state

十三、Agent 记忆条目表 agent_memory_item

context_summary 是摘要, agent_memory_item 是可检索的短期记忆条目。

```
create table agent_memory_item (  
  id text primary key,  
  
  user_id text not null references user_profile(user_id) on delete cascade,  
  
  memory_type text not null,  
  -- short_term / working / learning_preference / open_loop / warning /  
strategy  
  
  title text not null,  
  content_text text not null,  
  
  related_entity_type text,  
  -- question / knowledge_point / method / thinking_pattern / workflow  
  
  related_entity_id text,  
  related_entity_name text,  
  
  importance_score real not null default 0.5,  
  -- 0 到 1  
  
  freshness_score real not null default 1.0,  
  -- 随时间衰减  
  
  expires_at text,  
  -- 短期记忆可以过期  
  
  status text not null default 'active',  
  -- active / resolved / expired / deprecated  
  
  created_at text not null default current_timestamp,  
  updated_at text not null default current_timestamp,  
  
  metadata_json text not null default '{}'  
);
```

例子:

```
memory_type = open_loop
title = 夹逼法复习未闭合
content = 用户理解夹逼法结论，但对主动构造上下界仍不稳定。
related_entity_type = thinking_pattern
related_entity_name = 主动构造上下界
importance_score = 0.85
```

这类记忆可以直接进入 Prompt。

注意：这里存的是可观察的学习状态摘要，不是模型隐藏推理过程。

十四、当前上下文引用表 active_context_ref

Agent 运行时需要知道当前正在处理什么。

```
create table active_context_ref (
  id text primary key,

  user_id text not null references user_profile(user_id) on delete cascade,
  session_id text references learning_session(id) on delete cascade,

  ref_type text not null,
  -- active_question / active_attempt / active_workflow / active_topic /
  active_review

  ref_id text not null,

  ref_name text,

  status text not null default 'active',
  -- active / resolved / abandoned

  created_at text not null default current_timestamp,
  updated_at text not null default current_timestamp,

  unique(user_id, session_id, ref_type, ref_id)
);
```

用途：

当前正在讲哪道题
当前正在跑哪个复习 workflow

当前用户还没提交哪道题的答案
当前上下文里哪个问题没闭合

十五、系统运行状态表 `agent_runtime_state`

这张表存 Agent 当前系统状态，方便恢复、调试、前端展示。

```
create table agent_runtime_state (  
  id text primary key,  
  
  user_id text not null references user_profile(user_id) on delete cascade,  
  session_id text references learning_session(id) on delete cascade,  
  
  agent_name text not null,  
  -- main_agent / intent_planner / review_agent / context_compressor  
  
  state_key text not null,  
  state_value_json text not null,  
  
  status text not null default 'active',  
  -- active / superseded / cleared  
  
  created_at text not null default current_timestamp,  
  updated_at text not null default current_timestamp,  
  
  unique(user_id, session_id, agent_name, state_key)  
);
```

可以存：

```
{  
  "current_workflow": "REVIEW_AND_VARIANT",  
  "current_step": "WAITING_USER_ANSWER",  
  "active_question_id": "Q123",  
  "pending_state_delta_id": "D456"  
}
```

这和 `context_summary` 不同：

`context_summary` 给模型理解上下文。
`agent_runtime_state` 给系统恢复运行状态。

十六、Workflow 运行表 workflow_run

每次扣子 Workflow 运行都要记录。

```
create table workflow_run (  
  id text primary key,  
  
  user_id text not null references user_profile(user_id) on delete cascade,  
  session_id text references learning_session(id) on delete set null,  
  
  workflow_name text not null,  
  -- new_question_solve / review / method_transfer / context_compress  
  
  status text not null default 'running',  
  -- running / succeeded / failed / cancelled / waiting_user  
  
  input_json text not null default '{}',  
  output_json text not null default '{}',  
  
  current_step text,  
  
  error_message text,  
  
  started_at text not null default current_timestamp,  
  completed_at text,  
  
  metadata_json text not null default '{}'  
);
```

这张表用于回答：

这次 Agent 调用了哪个流程？
走到哪一步？
失败在哪？
是否等待用户作答？

十七、工具调用日志表 tool_call_log

工具调用必须记录。否则排查不了召回错误。

```
create table tool_call_log (  
  id text primary key,
```

```

workflow_run_id text references workflow_run(id) on delete cascade,

user_id text not null references user_profile(user_id) on delete cascade,
session_id text references learning_session(id) on delete set null,

tool_name text not null,
-- state.query_wrong_questions / asset.search_by_method /
state.write_attempt_result

input_json text not null default '{}',
output_summary_json text not null default '{}',

status text not null,
-- succeeded / failed / timeout / skipped

error_message text,

latency_ms integer,

created_at text not null default current_timestamp
);

```

注意： `output_summary_json` 不建议存完整大结果。完整题目资产应该在 PostgreSQL，需要时回查。

十八、待写回状态表 `pending_state_delta`

这是防止 Agent 幻觉污染状态库的关键设计。

主 Agent 可以提出状态变化建议，但不能直接写入最终状态。先进入 `pending_state_delta`，由规则层校验后再写入 `mastery_state`、`review_schedule` 等正式表。

```

create table pending_state_delta (
    id text primary key,

    user_id text not null references user_profile(user_id) on delete cascade,
    session_id text references learning_session(id) on delete set null,

    workflow_run_id text references workflow_run(id) on delete set null,
    attempt_id text references question_attempt(id) on delete set null,

    delta_type text not null,

```

```

-- mastery_update / error_pattern_update / review_schedule_update /
memory_update

proposed_by text not null default 'agent',
-- agent / rule / user / system

delta_json text not null,

validation_status text not null default 'pending',
-- pending / approved / rejected / applied

validation_reason text,

created_at text not null default current_timestamp,
validated_at text,
applied_at text
);

```

流程：

```

Agent 生成 state_delta
↓
写入 pending_state_delta
↓
规则层校验
↓
通过后更新 mastery_state / review_schedule / memory_item
↓
写 mastery_event / local_event_log

```

这样可以保证：

模型负责判断倾向。
 系统负责状态一致性。
 数据库只接受可验证变化。

这里要补上“可验证变化”最小校验条件，不然实现时还是会变成“写个待审核表但没人知道审什么”。

建议规则层至少校验：

1. 是否存在 `question_attempt` 或 `review_event` 等事实证据
2. 是否能拿到 `question_id` / `knowledge_point_id` / `method_id` 等引用
3. `proposed delta` 是否属于允许更新的字段集合

4. `mastery_change` / `priority_change` 是否在合法范围
5. 是否和当前 `workflow` 状态一致

例如：

用户提交了答案并已判错
→ 可以降低相关 `mastery_score`，提升对应 `review priority`

主 `Agent` 说“用户似乎已经理解”
但没有作答、没有证据
→ 只能写 `memory` / `summary`，不能改正式掌握状态

十九、本地事件日志 `local_event_log`

建议做轻量 event sourcing。

```
create table local_event_log (  
    id text primary key,  
  
    user_id text references user_profile(user_id) on delete cascade,  
    session_id text references learning_session(id) on delete set null,  
  
    event_type text not null,  
    -- attempt_created / mastery_updated / review_scheduled / memory_created  
    / workflow_failed  
  
    aggregate_type text,  
    -- attempt / mastery / review / memory / workflow  
  
    aggregate_id text,  
  
    event_json text not null default '{}',  
  
    created_at text not null default current_timestamp  
);
```

这张表的作用：

状态可回放
方便 `debug`
方便将来同步到云端
方便迁移 `PgSQL`

二十、系统 KV 表 system_kv

存系统级配置，不要乱塞业务数据。

```
create table system_kv (  
    key text primary key,  
  
    value_json text not null,  
  
    scope text not null default 'global',  
    -- global / user / session  
  
    updated_at text not null default current_timestamp  
);
```

可以存：

```
{  
  "max_context_turns": 6,  
  "context_compress_threshold_tokens": 3000,  
  "review_due_limit": 5,  
  "default_difficulty": 3  
}
```

二十一、推荐索引

```
create index idx_session_user_status  
on learning_session(user_id, status, started_at desc);  
  
create index idx_turn_session_time  
on conversation_turn(session_id, created_at);  
  
create index idx_attempt_user_question  
on question_attempt(user_id, question_id, created_at desc);  
  
create index idx_attempt_user_time  
on question_attempt(user_id, created_at desc);  
  
create index idx_attempt_tag_attempt  
on question_attempt_tag(attempt_id);
```

```

create index idx_attempt_tag_type_name
on question_attempt_tag(tag_type, tag_name);

create index idx_mastery_user_entity
on mastery_state(user_id, entity_type, entity_name);

create index idx_mastery_user_state
on mastery_state(user_id, state_label, updated_at desc);

create index idx_mastery_review_due
on mastery_state(user_id, next_review_at);

create index idx_error_user_status
on error_pattern_state(user_id, status, severity_score desc);

create index idx_review_user_due
on review_schedule(user_id, status, due_at, priority_score desc);

create index idx_context_user_scope
on context_summary(user_id, summary_scope, status, updated_at desc);

create index idx_memory_user_active
on agent_memory_item(user_id, status, importance_score desc, updated_at
desc);

create index idx_workflow_user_status
on workflow_run(user_id, status, started_at desc);

create index idx_tool_workflow
on tool_call_log(workflow_run_id, created_at);

create index idx_delta_status
on pending_state_delta(user_id, validation_status, created_at);

create index idx_event_user_time
on local_event_log(user_id, created_at desc);

```

二十二、Agent 启动时应该从 SQLite 读什么

主 Agent 不应该读全部历史。启动一轮任务时，建议 API 返回一个 `agent_bootstrap_context`：

```

{
  "user_snapshot": {

```

```
    "current_subject": "高等数学",
    "current_topic": "极限"
  },
  "session_summary": "用户最近在复习数列极限，夹逼法理解部分稳定，但主动构造上下界仍弱。",
  "active_context": {
    "active_question_id": "Q123",
    "current_workflow": "REVIEW_AND_VARIANT"
  },
  "weak_mastery_items": [
    {
      "entity_type": "thinking_pattern",
      "entity_name": "主动构造上下界",
      "mastery_score": 0.38,
      "state_label": "weak"
    }
  ],
  "due_reviews": [
    {
      "target_type": "method",
      "target_name": "夹逼估计",
      "priority_score": 0.86
    }
  ],
  "recent_error_patterns": [
    {
      "error_pattern_name": "不会从目标反推构造式",
      "severity_score": 0.74
    }
  ],
  "open_loops": [
    "需要做一道同知识点但换方法的极限题"
  ]
}
```

这就是 Agent 的短期记忆来源。

二十三、Prompt 拼接时 SQLite 的作用

主 Agent Prompt 里应拼接：

1. 用户当前学习快照
2. 当前 **session** 摘要
3. 最近薄弱知识点

4. 最近薄弱方法
5. 最近薄弱题型
6. 最近薄弱解题思想
7. 到期复习项
8. 当前 **active workflow**
9. 当前未闭合任务
10. 本轮用户输入

对应来源：

```
user_profile
context_summary
mastery_state
error_pattern_state
review_schedule
agent_runtime_state
active_context_ref
conversation_turn
```

不要拼接：

完整历史对话
完整题库
完整解析
完整工具日志

二十四、掌握度更新规则

第一版不需要复杂模型，先用规则即可。

每次 `question_attempt` 完成后：

1. 读取 `question_attempt_tag`
2. 对每个 `tag` 更新 `mastery_state`
3. 写 `mastery_event`
4. 如果错误明显，更新 `error_pattern_state`
5. 根据结果更新 `review_schedule`
6. 生成 `context_summary` / `agent_memory_item`

一个简单更新公式：

如果答对：

$$\text{new_score} = \text{old_score} + \alpha \times \text{weight} \times (1 - \text{old_score})$$

如果答错：

$$\text{new_score} = \text{old_score} - \beta \times \text{weight} \times \text{old_score}$$

建议：

$$\alpha = 0.08 \text{ 到 } 0.15$$

$$\beta = 0.10 \text{ 到 } 0.25$$

不同标签权重不同：

主知识点 $\text{weight} = 1.0$

辅助知识点 $\text{weight} = 0.5$

主方法 $\text{weight} = 1.0$

隐藏方法 $\text{weight} = 0.7$

解题思想 $\text{weight} = 0.9$

题型结构 $\text{weight} = 0.6$

状态标签规则可以先这样：

$\text{mastery_score} < 0.2$	unseen / exposed
$0.2 - 0.45$	weak
$0.45 - 0.65$	learning
$0.65 - 0.8$	reviewing / stable
> 0.8 且 $\text{confidence} > 0.7$	mastered

但注意：真正是否 mastered，不能只看分数，还要看 `confidence_score` 和题目多样性。

二十五、复习调度规则

复习优先级可以先由四个因素组成：

$$\text{priority} = \text{薄弱程度} + \text{错误频率} + \text{到期程度} + \text{重要性}$$

简单公式：

$$\begin{aligned} \text{priority_score} = \\ 0.35 \times (1 - \text{mastery_score}) \end{aligned}$$

```
+ 0.25 × severity_score
+ 0.25 × due_score
+ 0.15 × importance_score
```

复习间隔可以先用简化 SM-2 思想：

```
答错: interval_days = 1
部分掌握: interval_days = max(1, old_interval * 1.3)
答对: interval_days = old_interval * ease_factor
```

`ease_factor` 可随表现调整：

```
答错: ease_factor -= 0.2
部分掌握: ease_factor -= 0.05
答对: ease_factor += 0.05
```

这不需要一开始很精确，关键是表结构要能支持以后替换算法。

二十六、系统状态和学习状态必须分开

这是 SQLite 设计里非常重要的一点。

学习状态：

```
用户是否掌握夹逼法
用户是否容易算错
用户下次什么时候复习
用户最近薄弱题型是什么
```

系统状态：

```
当前 workflow 跑到哪一步
工具是否调用成功
是否等待用户提交答案
状态写回是否 pending
上下文是否已压缩
```

对应表：

```
学习状态：
mastery_state
```

```
error_pattern_state
review_schedule
question_attempt
review_event
```

系统状态:

```
agent_runtime_state
workflow_run
tool_call_log
pending_state_delta
local_event_log
system_kv
```

不要混在一张表里。否则后面会出现:

```
用户学会了某知识点
和
Workflow 已运行到 TOOL_CALLED
```

这两种完全不同的状态混杂在一起。

二十七、MVP 最小表结构

第一版不要一次做太重。建议先上 12 张表:

```
user_profile
learning_session
conversation_turn
question_attempt
question_attempt_tag
mastery_state
mastery_event
error_pattern_state
review_schedule
context_summary
workflow_run
tool_call_log
```

第一版先实现:

```
记录做题
记录错因
更新知识点掌握度
```

更新方法掌握度
更新题型 / 解题思想掌握度
生成 `session_summary`
查询薄弱项
查询到期复习
记录 `workflow` 和工具调用

暂时可以不做：

`agent_memory_item`
`active_context_ref`
`local_event_log`
复杂复习算法
多端同步

这里建议再收紧一下：

如果第一版要让 **Agent** 直接参与状态建议，
那么 `pending_state_delta` 不能省。

只有在下面这个更弱的 MVP 前提成立时，才可以暂时不建 `pending_state_delta`：

状态更新完全由确定性规则计算
Agent 不直接提出自由文本 `state_delta`

否则第一版就会出现业务风险：

主 **Agent** 的猜测直接污染长期学习状态

所以更稳妥的建议是：

第一版最小可行表结构里，把 `pending_state_delta` 提前纳入
`agent_memory_item` / `active_context_ref` / `local_event_log` 可以晚一阶段

二十八、SQLite API 设计

扣子不要直接访问 SQLite。还是通过你自己的 API 层。

建议提供这些接口：

```
state.get_agent_bootstrap_context(user_id, session_id)

state.create_session(user_id, session_type, subject, topic)

state.write_turn(session_id, role, content)

state.write_attempt_result(user_id, session_id, question_id, answer, result)

state.attach_attempt_tags(attempt_id, tags)

state.update_mastery_from_attempt(attempt_id)

state.query_weak_items(user_id, entity_type, limit)

state.query_due_reviews(user_id, limit)

state.write_review_event(user_id, schedule_id, result)

state.update_context_summary(user_id, session_id, summary)

state.write_agent_memory(user_id, memory_item)

system.create_workflow_run(user_id, workflow_name, input)

system.update_workflow_run(run_id, status, output)

system.write_tool_call(run_id, tool_name, input, output_summary, status)
```

Agent 只调用这些工具，不直接写 SQL。

二十九、一次完整状态写入示例

用户做了一道极限题，答错。PgSQL 返回这道题标签：

知识点：数列极限
方法：夹逼估计
题型结构：给定不等式求极限
解题思想：主动构造上下界

SQLite 写入流程：

1. `question_attempt` 新增一条记录
2. `question_attempt_tag` 写入四类标签

3. **mastery_state** 更新:
 - 数列极限下降
 - 夹逼估计下降
 - 给定不等式求极限下降
 - 主动构造上下界下降
4. **mastery_event** 写入四条变化事件
5. **error_pattern_state** 更新:
 - 不会主动构造上下界 **occurrence_count** + 1
6. **review_schedule** 新增或提前:
 - 主动构造上下界, 明天复习
 - 夹逼估计, 两天内复习
7. **context_summary** 更新:
 - 用户本轮暴露出夹逼结构识别不稳
8. **workflow_run** 更新为 **succeeded**
9. **tool_call_log** 记录所有工具调用

下一轮 Agent 启动时, 就能看到:

当前薄弱: 主动构造上下界
到期复习: 夹逼估计
建议召回: 同知识点但不同题面结构的极限题

三十、最终 SQLite 定位

最终你的 SQLite 应该能回答这些问题:

用户最近在学什么?
用户做过哪些题?
哪些题答错了?
错因是什么?
用户对某个知识点掌握到什么程度?
用户对某个方法掌握到什么程度?
用户对某类题型思想掌握到什么程度?
下一次该复习什么?
为什么该复习这个?
当前 **Agent** 工作流跑到哪一步?
上一次工具调用成功了吗?
当前 **Prompt** 应该拼接哪些短期记忆?
哪些状态变化还没被验证?

如果 SQLite 能回答这些问题, 它就不是普通缓存, 而是整个系统的 **状态中枢**。

最核心的设计判断是:

PgSQL 是题目资产网络。

SQLite 是用户学习状态机。

扣子 **Agent** 是状态机的调度器和解释器。

Iris 是状态变化的可视化界面。

这四层边界清楚，系统才不会乱。